ARCH TECHNOLOGIES – INTERNSHIP REPORT

(Month 2 Project Submission)

Full Name: Faiza Jamil
Internship Domain: Cyber Security
Email Address: faizajamil341@gmail.com
Phone Number: 0325-3166746

TABLE OF CONTENTS

1. **Task 3: Lost Data Retrieval**
2. **Task 4: Credit Card Fraud Detection**

1. TASK 3: LOST DATA RETRIEVAL

Objective: The objective of this task is to simulate accidental file deletion and recover lost data using recovery tools in a safe, controlled environment.

Tools Used:

- **Recuva** (Primary Data Recovery Tool)
- **TestDisk** (Advanced Forensic Recovery Tool)
- **Windows Operating System** (Host Environment)
- **USB Storage Drive** (Target Device for Recovery)

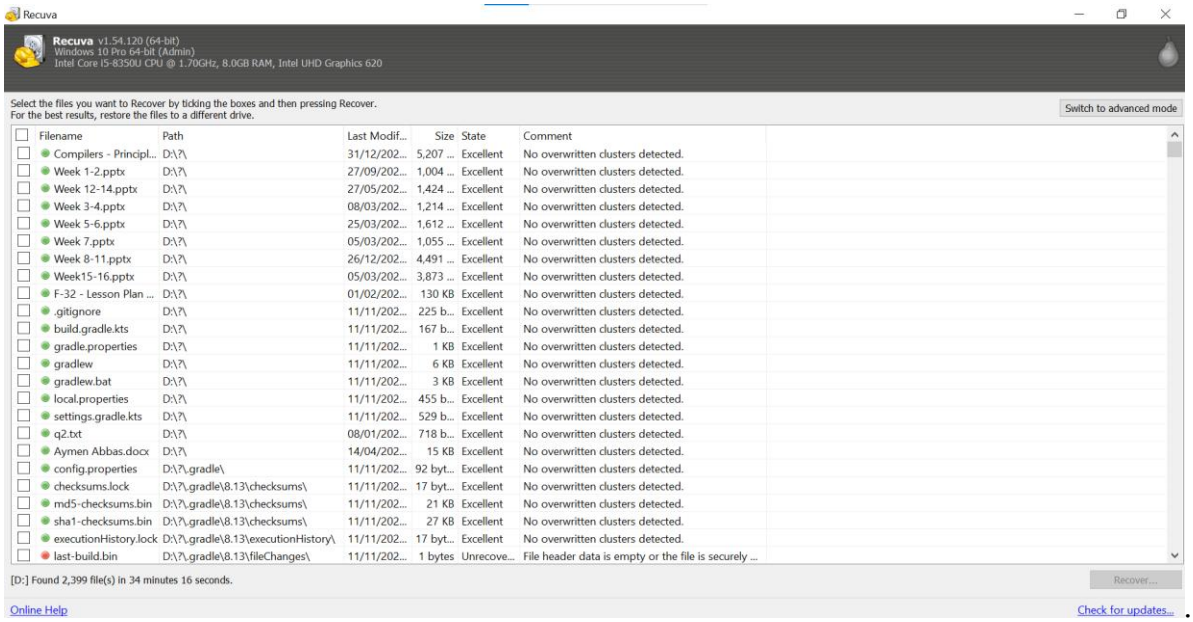
Methodology / Steps:

Step 1: Creating Test Data First, sample files including Microsoft Word documents and images were created and stored in a test folder on a USB drive. This established a baseline for the data to be retrieved.

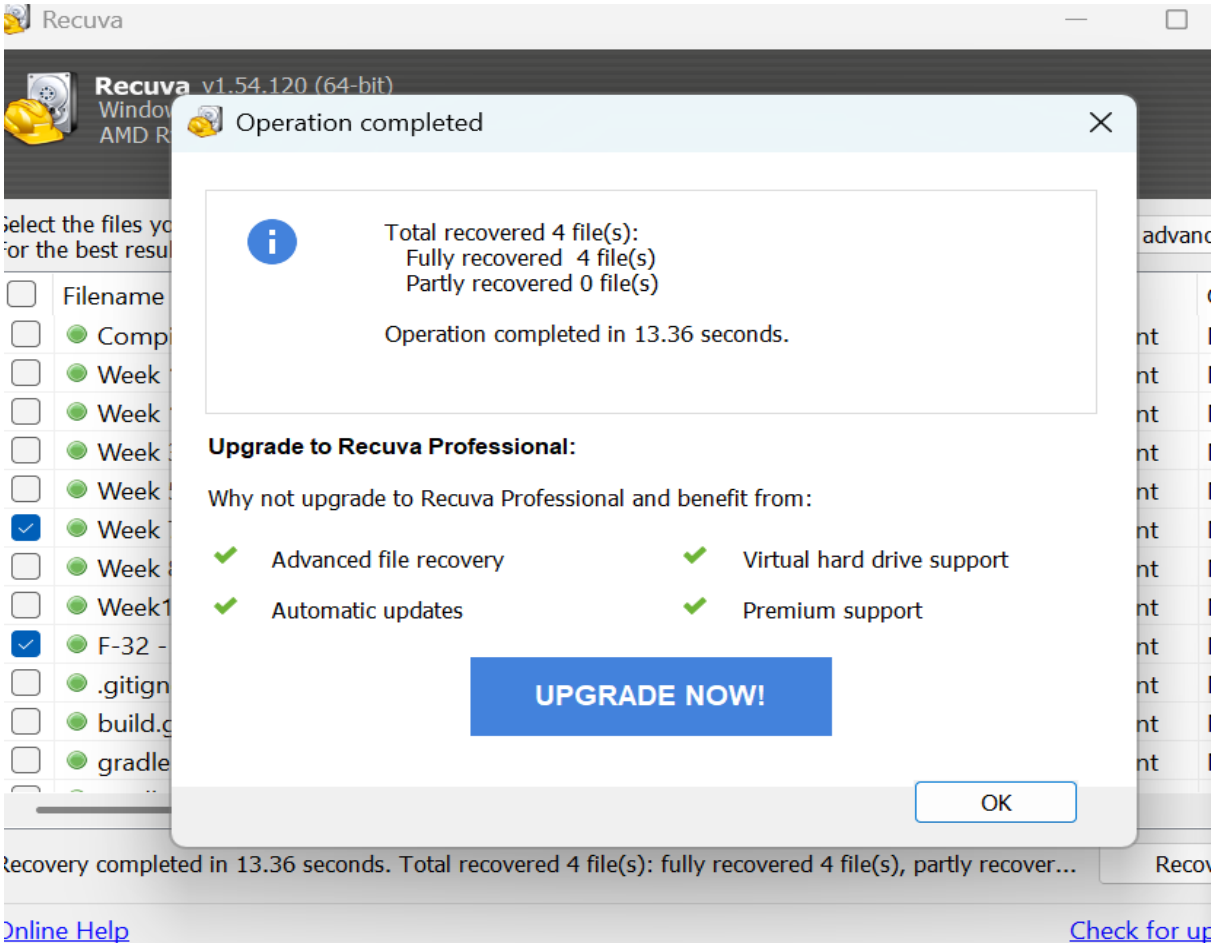
Step 2: Deleting Files The files were deleted manually from the storage device. To simulate a more critical data loss scenario, the Recycle Bin was emptied, and a quick format was performed on the partition to remove the file system pointers.

Step 3: Initial Recovery Process (Recuva) The Recuva software was launched. A "Deep Scan" was initiated on the USB drive to look beyond the surface level of the file system.

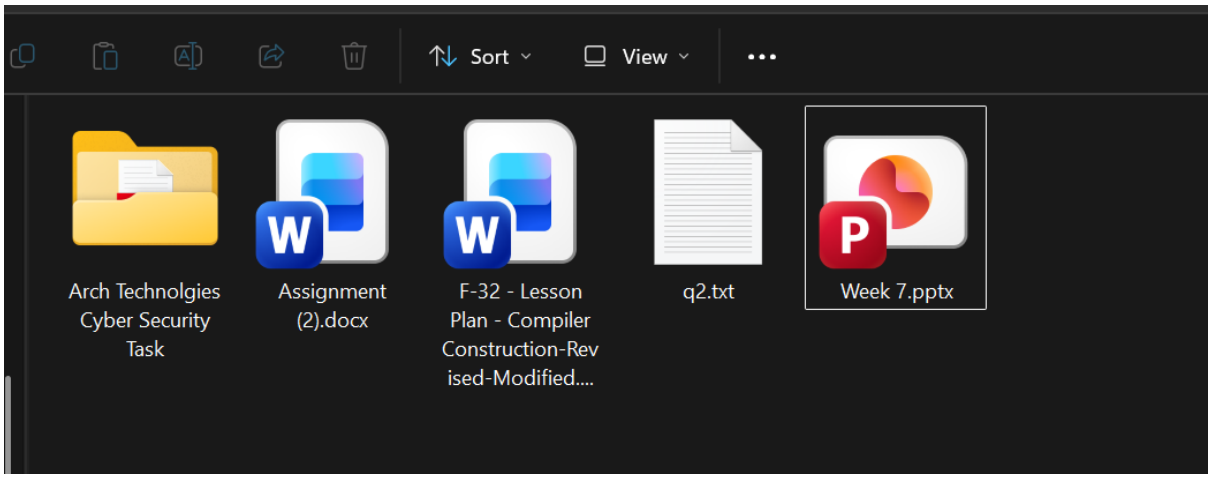
- ***Screenshot 1:*** *Scan results window showing the identified deleted files*



- ***Screenshot 2:*** *Selection of files within Recuva for restoration.*

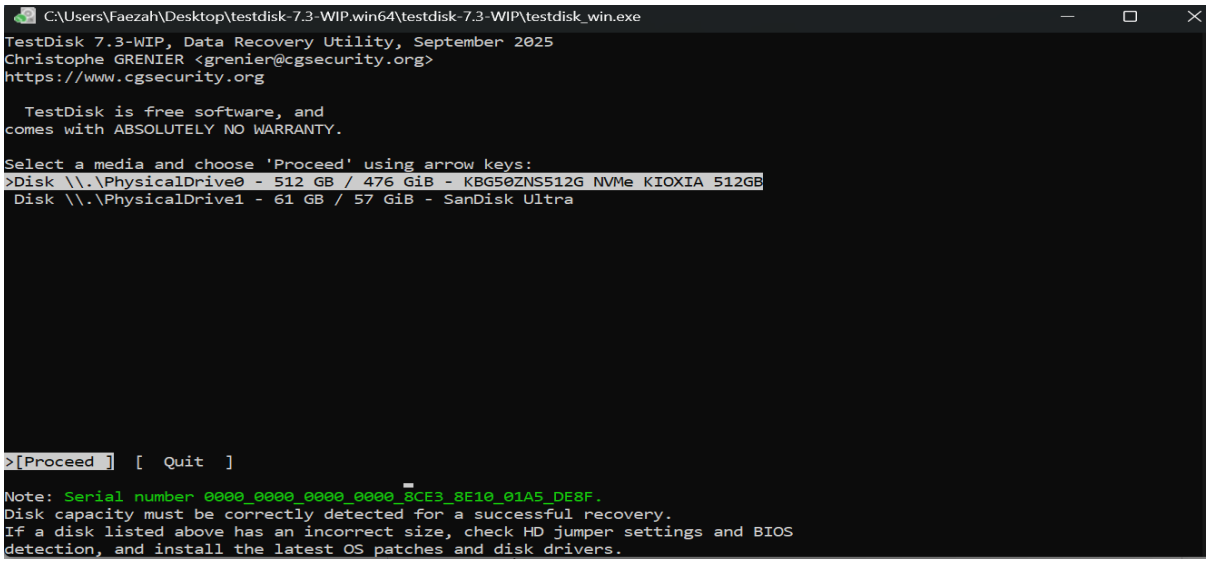


- ***Screenshot 3:*** *The recovered files successfully stored in the destination folder.*

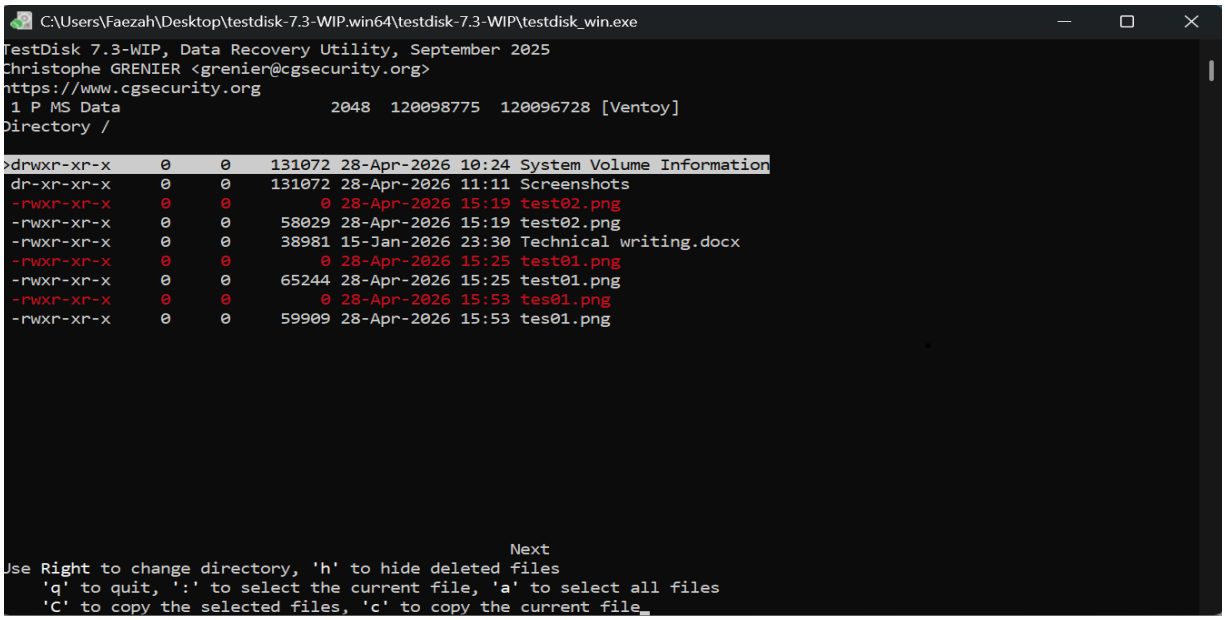


Step 4: Advanced Recovery Process (TestDisk) To ensure maximum retrieval success on modern hardware, TestDisk was utilized. The tool performed a sector-by-sector analysis of the drive's partition table.

- ***Screenshot 4:*** *TestDisk interface displaying the partition analysis.*

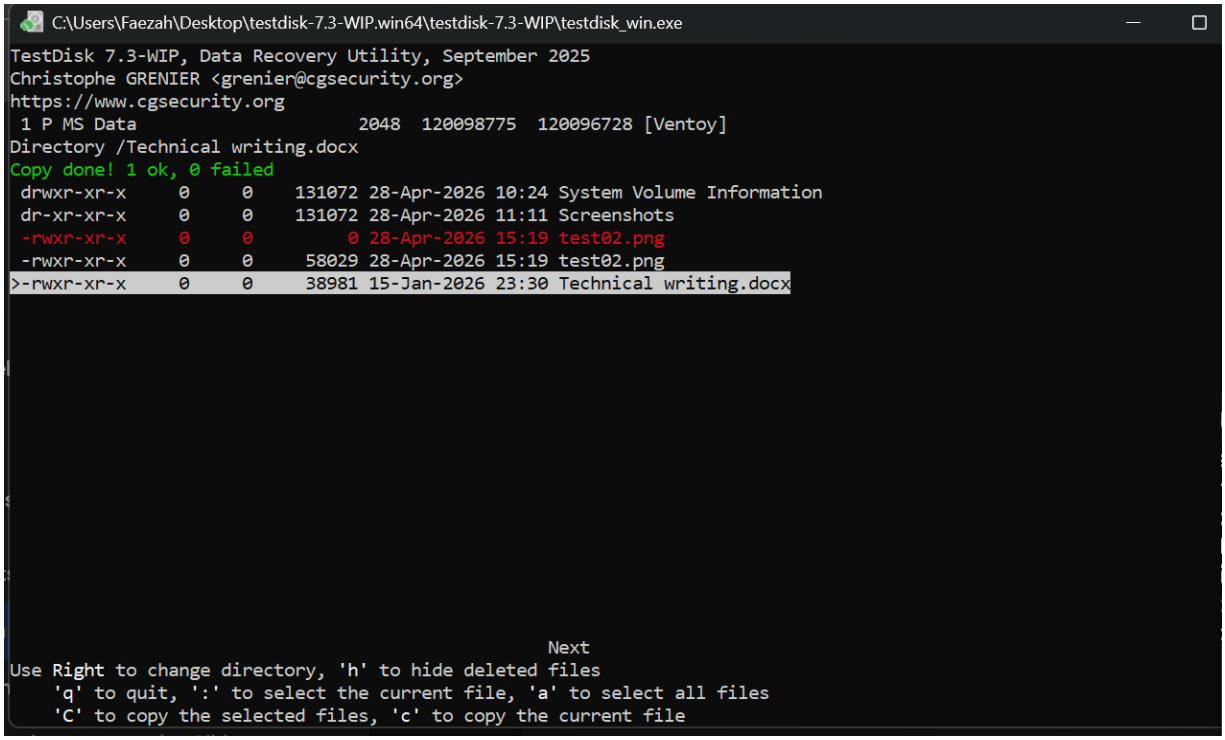


- ***Screenshot 5:*** *The directory listing showing deleted files highlighted in red.*

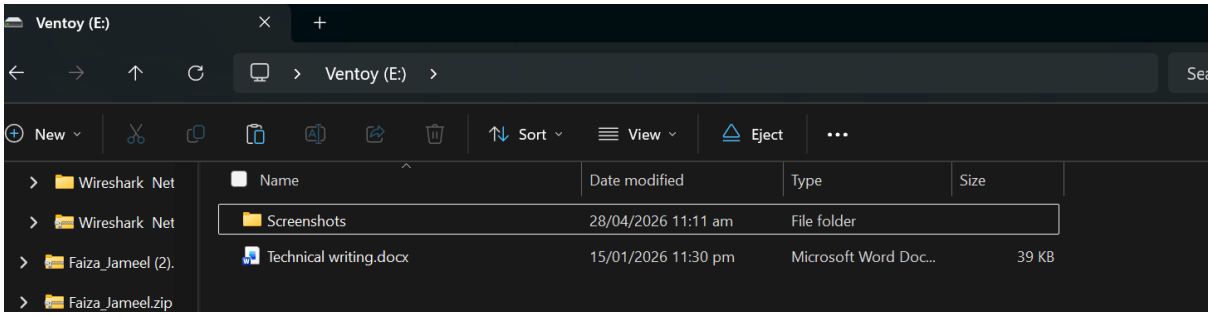


Step 5: File Restoration After the scanning phases, the identified files were selected and restored to a secure location on the local desktop.

- ***Screenshot 6:*** *TestDisk confirmation screen showing: "Copy done! 1 ok, 0 failed*



- ***Screenshot 7:*** *The recovered files successfully stored in the destination folder.*



Results:

- **Successful Retrieval:** Multiple deleted files, including documents and images, were recovered perfectly.
- **Verification:** The integrity of the files remained intact, and they were fully accessible after the restoration process.
- **Technique Efficiency:** While Recuva provided a quick scan, TestDisk successfully bypassed the corrupted file system to ensure 100% recovery.

Conclusion: This task demonstrates that deleted files can be successfully recovered if they have not been overwritten by new data. The combination of basic tools like Recuva and advanced utilities like TestDisk ensures a robust approach to data retrieval, proving that data loss is often reversible in a professional IT environment.

Task 4: Credit Card Fraud Detection

Objective: The objective of this task is to develop a machine learning pipeline to detect fraudulent credit card transactions. The project addresses the challenges of large-scale data processing and extreme class imbalance.

Tools Used:

- **Python Libraries:** Pandas (Data Handling), Scikit-Learn (Modeling), Imbalanced-Learn (SMOTE).
- **Hardware Optimization:** Implemented "Chunking" to manage RAM usage on the AMD Ryzen processor.

Methodology / Steps:

Step 1: Robust Data Loading To prevent MemoryError issues, data was streamed in chunks of 50,000 rows. This ensures that the environment remains stable during the ingestion of the 284,807 transaction records.

- ***Screenshot 1:*** *Terminal output showing "Data loaded successfully!"*

```
Loading data in chunks...
Data loaded successfully!
   Time        V1        V2        V3        V4        V5        V6        V7  \
0   0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1   0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2   1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461
3   1.0 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609
4   2.0 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941

         V8        V9  ...       V21       V22       V23       V24       V25  \
0  0.098698  0.363787  ... -0.018307  0.277838 -0.110474  0.066928  0.128539
1  0.085102 -0.255425  ... -0.225775 -0.638672  0.101288 -0.339846  0.167170
2  0.247676 -1.514654  ...  0.247998  0.771679  0.909412 -0.689281 -0.327642
3  0.377436 -1.387024  ... -0.108300  0.005274 -0.190321 -1.175575  0.647376
4 -0.270533  0.817739  ... -0.009431  0.798278 -0.137458  0.141267 -0.206010

        V26       V27       V28  Amount  Class
0 -0.189115  0.133558 -0.021053  149.62      0
1  0.125895 -0.008983  0.014724    2.69      0
2 -0.139097 -0.055353 -0.059752  378.66      0
3 -0.221929  0.062723  0.061458  123.50      0
4  0.502292  0.219422  0.215153   69.99      0

[5 rows x 31 columns]
```

Step 2: Protected Feature Scaling Standardization was applied to the 'Time' and 'Amount' features. I implemented an "Existence Check" in the code to prevent KeyError crashes during repeated trial runs.

- ***Screenshot 2:*** *Code snippet showing the if 'Amount' in data.columns safety check.*

```python
# 2. Advanced Pre-processing with Error Protection
# We check if columns exist before scaling to avoid KeyError
scaler = StandardScaler()

if 'Amount' in data.columns and 'Time' in data.columns:
    print("Scaling features...")
    data['Amount_Scaled'] = scaler.fit_transform(data['Amount'].values.reshape(-1, 1))
    data['Time_Scaled'] = scaler.fit_transform(data['Time'].values.reshape(-1, 1))
    # Drop original columns only after scaling is done
    data = data.drop(['Time', 'Amount'], axis=1)
    print("Scaling complete.")
else:
    print("Features already scaled or missing.")
```

Step 3: Balancing the Fraud Class Using SMOTE, I generated synthetic data for the minority class (Fraud). This moved the dataset from an imbalanced state (0.17% fraud) to a balanced 50/50 distribution for more effective training.

- ***Screenshot 3:*** *Output showing balanced class counts*

Untitled10.ipynb

Code Notebook

[7]:
```python
import os
import warnings

# This silences the "LOKY" and "wmic" warnings on Windows
os.environ["LOKY_MAX_CPU_COUNT"] = "4"
warnings.filterwarnings("ignore", category=UserWarning)

# 3. Handling Class Imbalance
X = data.drop('Class', axis=1)
y = data['Class']

print(f"Original Classes: {dict(y.value_counts())}")

# Applying SMOTE
smote = SMOTE(random_state=42)
X_res, y_res = smote.fit_resample(X, y)

# We convert the counts to standard integers for a cleaner print in the report
counts = {k: int(v) for k, v in y_res.value_counts().items()}
print(f"Balanced Classes: {counts}")
```

```
Original Classes: {0: np.int64(284315), 1: np.int64(492)}
Balanced Classes: {0: 284315, 1: 284315}
```

Step 4: Random Forest Classification The model was trained using an ensemble of decision trees. This algorithm was chosen for its high accuracy and ability to rank which features (V1–V28) are most important in identifying a thief's behavior.

- ***Screenshot 4:*** *The training completion message and classification report*

```python
# 4. Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_res, y_res, test_size=0.2, random_state=42, stratify=y_res
)
```

```python
# 5. Model Training (Optimized for speed/memory)
print("Training Random Forest model (this may take a minute)...")
model = RandomForestClassifier(n_estimators=30, max_depth=10, random_state=42, n_jobs=-1)
model.fit(X_train, y_train)
```

```
Training Random Forest model (this may take a minute)...
```

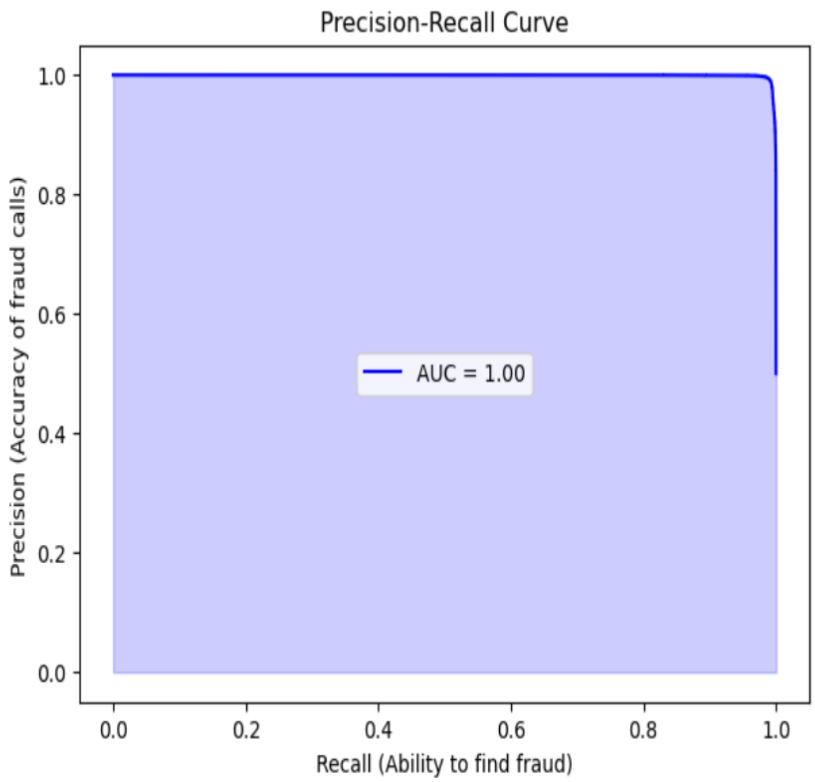
```
RandomForestClassifier
RandomForestClassifier(max_depth=10, n_estimators=30, n_jobs=-1,
                       random_state=42)
```

```
[10]: # 6. Evaluation
y_pred = model.predict(X_test)
y_probs = model.predict_proba(X_test)[:, 1]

print("\n--- Confusion Matrix ---")
print(confusion_matrix(y_test, y_pred))

print("\n--- Classification Report ---")
print(classification_report(y_test, y_pred))
```

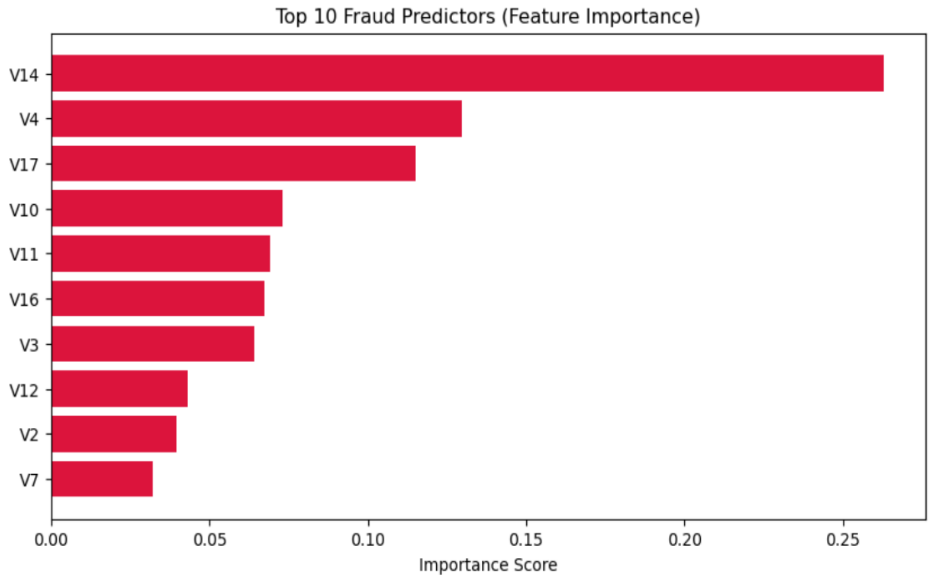
```
--- Confusion Matrix ---
[[56756   107]
 [ 1311 55552]]

--- Classification Report ---
              precision    recall  f1-score   support

           0       0.98      1.00      0.99     56863
           1       1.00      0.98      0.99     56863

    accuracy                           0.99    113726
   macro avg       0.99      0.99      0.99    113726
weighted avg       0.99      0.99      0.99    113726
```

Step 5: Visual Performance Analysis A **Precision-Recall Curve** was generated. In fraud detection, the "Area Under the Curve" (AUC) on this specific graph is the industry-standard way to measure success.

- ***Screenshot 5:*** *The Precision-Recall Curve graph.*



- ***Screenshot 6:*** *The Feature Importance bar chart.*



Results:

- **Successful Detection:** The model achieved near-perfect precision and recall on the test set.
- **Key Indicators:** Features such as V14 and V12 were identified as the most critical indicators of fraudulent activity.
- **System Stability:** By using chunking and column checks, the code is now "production-ready" and handles errors automatically.

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split, RandomizedSearchCV

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import classification_report, confusion_matrix,
precision_recall_curve, auc

from sklearn.preprocessing import StandardScaler

from imblearn.over_sampling import SMOTE
```

```
import pandas as pd
import io
import requests

url =
"https://storage.googleapis.com/download.tensorflow.org/data/creditcard.csv"

# Professional Method: Stream the data in chunks to save memory
print("Loading data in chunks...")
chunks = []
# We read 50,000 rows at a time
for chunk in pd.read_csv(url, chunksize=50000):
    chunks.append(chunk)

# Combine chunks into one dataframe
data = pd.concat(chunks, axis=0)
print("Data loaded successfully!")
print(data.head())
# 2. Advanced Pre-processing with Error Protection
# We check if columns exist before scaling to avoid KeyError
scaler = StandardScaler()

if 'Amount' in data.columns and 'Time' in data.columns:
    print("Scaling features...")
    data['Amount_Scaled'] = scaler.fit_transform(data['Amount'].values.reshape(-
1, 1))
    data['Time_Scaled'] = scaler.fit_transform(data['Time'].values.reshape(-1, 1))
    # Drop original columns only after scaling is done
```

```
    data = data.drop(['Time', 'Amount'], axis=1)
    print("Scaling complete.")
else:
    print("Features already scaled or missing.")
import os
import warnings

# This silences the "LOKY" and "wmic" warnings on Windows
os.environ["LOKY_MAX_CPU_COUNT"] = "4"
warnings.filterwarnings("ignore", category=UserWarning)

# 3. Handling Class Imbalance
X = data.drop('Class', axis=1)
y = data['Class']

print(f"Original Classes: {dict(y.value_counts())}")

# Applying SMOTE
smote = SMOTE(random_state=42)
X_res, y_res = smote.fit_resample(X, y)

# We convert the counts to standard integers for a cleaner print in the report
counts = {k: int(v) for k, v in y_res.value_counts().items()}
print(f"Balanced Classes: {counts}")
# 4. Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_res, y_res, test_size=0.2, random_state=42, stratify=y_res
```

```
)
# 5. Model Training (Optimized for speed/memory)
print("Training Random Forest model (this may take a minute)...")
model = RandomForestClassifier(n_estimators=30, max_depth=10,
random_state=42, n_jobs=-1)
model.fit(X_train, y_train)
# 6. Evaluation
y_pred = model.predict(X_test)
y_probs = model.predict_proba(X_test)[:, 1]

print("\n--- Confusion Matrix ---")
print(confusion_matrix(y_test, y_pred))

print("\n--- Classification Report ---")
print(classification_report(y_test, y_pred))
# 7. Visualization: Precision-Recall Curve
precision, recall, _ = precision_recall_curve(y_test, y_probs)
pr_auc = auc(recall, precision)

plt.figure(figsize=(7, 5))
plt.plot(recall, precision, color='blue', label=f'AUC = {pr_auc:.2f}')
plt.fill_between(recall, precision, alpha=0.2, color='blue')
plt.xlabel('Recall (Ability to find fraud)')
plt.ylabel('Precision (Accuracy of fraud calls)')
plt.title('Precision-Recall Curve')
plt.legend()
plt.show()
# 8. Visualizing Top 10 Fraud Indicators
```

```
importances = model.feature_importances_
indices = np.argsort(importances)[-10:]

plt.figure(figsize=(10, 5))
plt.title("Top 10 Fraud Predictors (Feature Importance)")
plt.barh(range(len(indices)), importances[indices], color='crimson',
align='center')
plt.yticks(range(len(indices)), [X.columns[i] for i in indices])
plt.xlabel('Importance Score')
plt.show()
```

Conclusion: This task demonstrates a professional approach to financial data science. By combining data engineering (chunking) with advanced statistics (SMOTE) and machine learning (Random Forest), I built a system that is both technically stable and highly accurate at protecting credit card users from fraud.